Imbalanced Classification for Vehicle Insurance Fraud Detection

Gurman Avtar (BSD-CC-2408)
Ridhwan Choudhari (BSD-CC-2417)
Supratik Biswas (BSD-CC-2423)

Abstract

Fraudulent claims make up only about 6% of the 15,420 vehicle insurance claims in the public dataset we used, so a classifier that ignores the minority class can still appear highly accurate. In this project we compared Logistic Regression, Decision Tree and Random Forest classifiers, each trained without any imbalance handling and with four imbalance-handling techniques: class weighting, random oversampling, SMOTE (implemented using SMOTENC) and random undersampling. All resampling was applied to the training folds only, to avoid data leakage. The 15 model and strategy combinations were compared using 5-fold stratified cross-validation repeated three times, and then evaluated on a held-out 20% test set.

Without imbalance handling, all three models flagged almost no fraud at the default 0.5 threshold (test recall between 0 and 0.05). Every imbalance-handling technique improved recall and F1-score in all 15 cross-validation folds. The selected model, a Random Forest with class weighting, detected 110 of the 185 fraudulent claims in the test set (recall 0.595, F1-score 0.281), but at a false-positive rate of 16.8%. SMOTE improved on the baseline but was outperformed by class weighting and random oversampling. We also found that imbalance handling did not clearly improve the threshold-free metrics (PR-AUC and ROC-AUC).

Table of Contents

1 Introduction **3**

2 Dataset **3**

3 Exploratory data analysis **4**

3.1 Class imbalance 4

3.2 Missing values and data quality 4

3.3 Relationship between the features and fraud 4

3.4 Redundant features 7

4 Methodology **7**

4.1 Data split 8

4.2 Preprocessing 8

4.3 Classifiers 8

4.4 Imbalance-handling strategies 8

4.5 Avoiding data leakage 9

4.6 Evaluation metrics 9

4.7 Model selection and testing 10

5 Results **10**

5.1 Cross-validation comparison 10

5.2 Effect of imbalance handling 11

5.3 SMOTE compared with the baseline 11

5.4 Selected model 12

5.5 Results on the test set 12

6 Discussion **14**

7 Conclusion **14**

Appendix: Supporting files and reproducibility **14**

1 Introduction

Insurance fraud is a costly problem for insurers, and detecting fraudulent claims can be framed as a binary classification task: given the details of a claim, the policy and the accident, predict whether the claim is fraudulent or legitimate. The difficulty is that fraud is rare. In our dataset only 923 of 15,420 claims (5.99%) are fraudulent. A model that simply labels every claim as legitimate would be 94% accurate while catching no fraud at all, so accuracy is not a useful measure here.

Standard classifiers are usually trained to minimise overall error, which pushes them towards the majority class. Several techniques have been proposed to counter this, either by changing the training data (oversampling the minority class, undersampling the majority class, or generating synthetic minority examples with SMOTE) or by changing the learning algorithm (giving more weight to errors on the minority class).

The objectives of this project were to:

1. Explore and preprocess the data, examining class imbalance, missing values and relationships between the features and fraud.
2. Build baseline classifiers: Logistic Regression, Decision Tree and Random Forest.
3. Apply class weighting, random oversampling, SMOTE and random undersampling, using them on training data only to avoid data leakage.
4. Compare the models using precision, recall, F1-score, PR-AUC, ROC-AUC and confusion matrices, with an emphasis on detecting fraud while controlling false positives.
5. Identify a model and imbalance-handling strategy that gives a suitable balance between fraud detection, false-positive control and overall performance.

We expected the imbalance-handling techniques to improve detection of the minority class, but we treated this as something to test rather than assume. Section 2 describes the dataset, Section 3 the exploratory analysis and Section 4 the methodology. The results are given in Section 5, followed by a discussion in Section 6 and conclusions in Section 7.

2 Dataset

We used the publicly available Vehicle Insurance Claim Fraud Detection dataset, which can be downloaded from Kaggle at `https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection` (file `fraud_oracle.csv`).

The dataset contains 15,420 claims made between 1994 and 1996, described by 33 columns. The target variable is `FraudFound_P` (1 = fraud, 0 = legitimate). `PolicyNumber` is a unique identifier and was not used as a feature. Most of the remaining variables are categorical, and several numeric quantities are stored as ranges in text form (for example `VehiclePrice` = "20000 to 29000" or `Days_Policy_Claim` = "more than 30"). Table 1 groups the variables.

The class distribution is highly imbalanced: 14,497 legitimate claims (94.01%) and 923 fraudulent claims (5.99%), a ratio of about 15.7 to 1.

Table 1: Variables in the dataset.

Group	Variables
Accident timing	Month, WeekOfMonth, DayOfWeek
Claim timing	MonthClaimed, WeekOfMonthClaimed, DayOfWeekClaimed, Days_Policy_Accident, Days_Policy_Claim
Policy holder	Sex, MaritalStatus, Age, AgeOfPolicyHolder, DriverRating, PastNumberOfClaims, AddressChange_Claim, NumberOfCars
Vehicle	Make, VehicleCategory, VehiclePrice, AgeOfVehicle
Policy	PolicyType, BasePolicy, Deductible, AgentType, RepNumber, Year
Accident details	AccidentArea, Fault, PoliceReportFiled, WitnessPresent, NumberOfSuppliments
Identifier and target	PolicyNumber (not used), FraudFound_P (target)

3 Exploratory data analysis

3.1 Class imbalance

Figure 1(a) shows the class distribution. With a ratio of 15.7 legitimate claims for every fraudulent one, a random classifier would have a precision of only about 0.06. This is also the PR-AUC of a model with no skill, which we use as a reference point in Section 5.

3.2 Missing values and data quality

The dataset has no explicitly missing values, but some entries are clearly placeholders:

- `Age` is 0 for 320 claims (2.08%). All of these claims have `AgeOfPolicyHolder` = "16 to 17", so it seems age was simply not recorded for the youngest policy holders. We set these values to missing and kept an indicator variable `AgeWasZero`.
- `MonthClaimed` and `DayOfWeekClaimed` are "0" for one claim each, and were also set to missing.
- There are no duplicate rows (ignoring `PolicyNumber`).

Missing values were imputed inside each training fold (median for numeric features, most frequent level for categorical features), so that information from the validation and test data was never used. `Year` and `RepNumber` are stored as numbers but were treated as categorical, since they are labels rather than quantities.

3.3 Relationship between the features and fraud

For the categorical features we used the bias-corrected Cramér's V together with chi-square tests, adjusting the p-values for multiple testing with the Benjamini-Hochberg method. For features with sparse categories (expected counts below 5) we report Cramér's V only. The strongest associations were with `PolicyType` (V = 0.167), `BasePolicy` (0.161), `VehicleCategory` (0.137) and `Fault` (0.131), followed by `AddressChange_Claim` (0.081), `VehiclePrice` (0.064) and `PastNumberOfClaims` (0.057). Figure 1(b) shows the

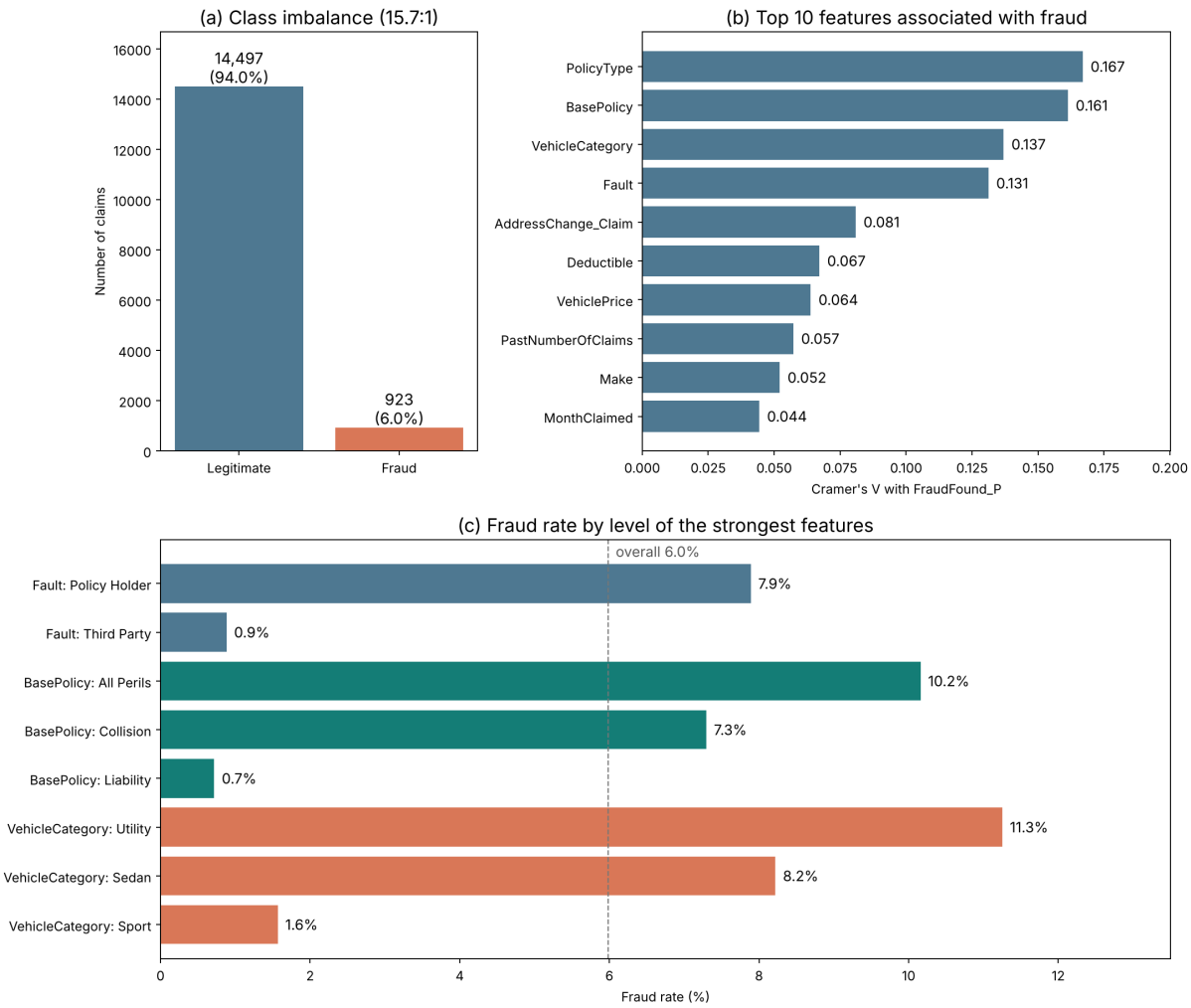


Figure 1: Key EDA plots: (a) class imbalance, (b) the ten features most associated with fraud (Cramér's V), (c) fraud rate by level of the three strongest features.

ten strongest associations. All of these are weak to moderate, which suggests that no single feature separates fraudulent claims from legitimate ones.

The numeric features are even more weakly related to fraud. The largest Spearman correlations were for `Deductible` ($\rho = 0.041$, adjusted $q < 0.001$), `AgeWasZero` (0.023) and `Age` (-0.022). `WeekOfMonth`, `DriverRating` and `WeekOfMonthClaimed` were not significant. Fraudulent claimants were slightly younger (median age 37 against 39; Mann-Whitney $p = 0.0065$), but the fraud rate only varies between 4.9% and 7.1% across age bands.

Table 2 and Figure 1(c) show the fraud rate for the levels of the more informative features. The overall fraud rate is 6.0%.

Table 2: Fraud rate by selected feature levels (highest and lowest level for each feature).

Feature	Level	Claims	Fraud rate (%)
Fault	Policy Holder	11,230	7.9
Fault	Third Party	4,190	0.9
PolicyType	Sport - Collision	348	13.8
PolicyType	Sedan - Liability	4,987	0.7
VehicleCategory	Utility	391	11.3
VehicleCategory	Sport	5,358	1.6
VehiclePrice	less than 20000	1,096	9.4
VehiclePrice	60000 to 69000	87	4.6
AccidentArea	Rural	1,598	8.3
AccidentArea	Urban	13,822	5.7
PastNumberOfClaims	none	4,352	7.8
PastNumberOfClaims	more than 4	2,010	3.4

The clearest difference is for `Fault`: when the policy holder was at fault, the fraud rate was 7.9%, compared with only 0.9% when a third party was at fault. Liability-only policies also have very little fraud (0.7%).

3.4 Redundant features

Figure 2 shows the correlation heatmap of all the features and the target. Since most features are categorical, we used Cramér's V for every pair; the numeric features take only a few values and were treated as categories, and `Age` was grouped into age bands. The features are ordered by their association with fraud, so the most relevant ones are in the top-left corner. `PolicyType` is almost perfectly associated with both `VehicleCategory` and `BasePolicy` ($V \approx 1.00$), because a policy type such as "Sedan - Collision" is just a combination of the vehicle category and the base policy. The two age variables, `Age` and `AgeOfPolicyHolder`, are also strongly related ($V = 0.90$), as are `Month` and `MonthClaimed` ($V = 0.75$), since most claims are made soon after the accident. We kept all the features, because tree-based models are not affected by this redundancy. However, it means the individual Logistic Regression coefficients for these features should not be interpreted.

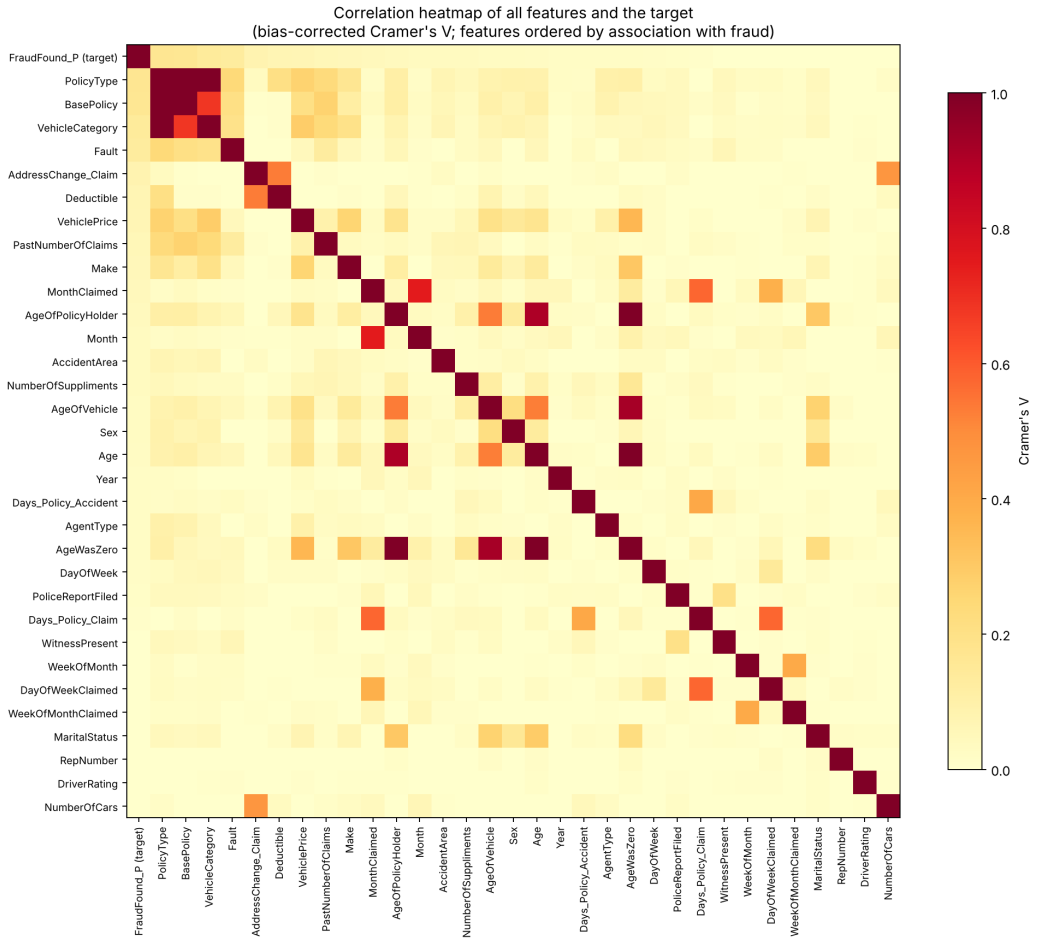


Figure 2: Correlation heatmap of all features and the target (bias-corrected Cramér's V).

These associations are descriptive only and do not imply that any feature causes fraud.

4 Methodology

The overall procedure was as follows:

1. Split the data once into a development set (80%) and a held-out test set (20%), stratified by the target.
2. On the development set, run 5-fold stratified cross-validation repeated 3 times (15 folds). In each fold, fit the preprocessing, the imbalance-handling step and the classifier on the training part only, and evaluate on the validation part.
3. Select the model and imbalance strategy with the highest mean cross-validated F1-score.
4. Refit every candidate on the full development set and evaluate it on the test set.

4.1 Data split

The stratified split gave a development set of 12,336 claims (738 fraudulent) and a test set of 3,084 claims (185 fraudulent), both with about 6% fraud. The test set was not used for choosing any model or imbalance strategy.

4.2 Preprocessing

Numeric features were imputed with the median and standardised. Categorical features were imputed with the most frequent level and then one-hot encoded. Range variables such as `VehiclePrice` and `AgeOfVehicle` were treated as categorical rather than converted to numbers, so the models could learn a separate effect for each range. Because SMOTENC needs to know which columns are categorical, the categorical features were first coded as integers, the resampling step was applied, and only then were they one-hot encoded for the classifier.

4.3 Classifiers

We used three classifiers of increasing flexibility:

- **Logistic Regression** with L2 regularisation.
- **Decision Tree** with a maximum depth of 7 and at least 30 claims per leaf, to limit overfitting.
- **Random Forest** with 200 trees and at least 5 claims per leaf.

4.4 Imbalance-handling strategies

Each classifier was trained with five strategies (Table 3), giving 15 candidates in total.

SMOTE creates a synthetic minority example by taking a fraud case, picking one of its k nearest fraud neighbours, and placing a new point at a random position on the line between them:

$$x_{\text{new}} = x + \lambda\,(z - x), \qquad \lambda \sim U(0, 1),$$

where x is a fraud case and z is one of its k nearest fraud neighbours.

Table 3: Imbalance-handling strategies.

Strategy	What it does to the training data	Implementation
No resampling (baseline)	Nothing; trained on the original 15.7:1 data	–
Class weighting	Data unchanged; errors on fraud cases weighted about 8.4 and on legitimate cases about 0.53	`class_weight="balanced"`
Random oversampling	Fraud cases duplicated at random until the classes are equal	`RandomOverSampler`
SMOTE	New synthetic fraud cases created until the classes are equal	`SMOTENC` ($k = 5$)
Random undersampling	Legitimate cases removed at random until the classes are equal	`RandomUnderSampler`

This interpolation only makes sense for numeric features. Most of our features are categorical, and interpolating between category codes would give meaningless values. We therefore implemented SMOTE using SMOTENC, the version for mixed numeric and categorical data. It interpolates the numeric features as above, and gives each categorical feature the most frequent value among the nearest neighbours.

4.5 Avoiding data leakage

If resampling is done before splitting the data, duplicated or synthetic fraud cases built from validation claims can end up in the training data, which makes the results look better than they really are. To prevent this, every step (imputation, scaling, encoding, resampling and the classifier) was placed in a single imbalanced-learn `Pipeline`. The resampler is only applied when the pipeline is fitted, so it only ever sees the training part of each fold. Validation and test data keep their natural 6% fraud rate.

4.6 Evaluation metrics

Treating fraud as the positive class, with TP, FP, TN and FN the counts in the confusion matrix, the main metrics are:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \qquad \text{FPR} = \frac{FP}{FP + TN}.$$

Recall measures how much fraud is caught, and precision and the false-positive rate (FPR) measure how many legitimate claims are wrongly flagged. These depend on the decision threshold, which we set to the default of 0.5. We also report two threshold-free metrics that measure how well a model ranks fraudulent claims above legitimate ones: ROC-AUC, and PR-AUC computed as average precision, which is more informative than ROC-AUC when the positive class is rare.

4.7 Model selection and testing

The overall winner was the candidate with the highest mean cross-validated F1-score, with PR-AUC used to break ties. We chose F1 because it balances fraud detection (recall) against the share of flagged claims that are actually fraud (precision). Since precision falls when many legitimate claims are flagged, F1 also penalises models with too many false positives, and we report the false-positive rate of every candidate alongside it.

After selection, all 15 candidates were refitted on the full development set and evaluated on the test set for reporting. The test results were not used to choose anything. To measure uncertainty on the test set we used 1,000 stratified bootstrap resamples to get 95% confidence intervals for the selected model's metrics and for its improvement over the same model without imbalance handling.

5 Results

5.1 Cross-validation comparison

Table 4 gives the cross-validated results for all 15 candidates at the default threshold of 0.5, and Figure 3 shows the same comparison graphically.

Table 4: Repeated stratified cross-validation results (mean over 15 folds, threshold 0.5). Standard deviations are shown in Figure 3.

Model	Strategy	Precision	Recall	F1-score	FPR	PR-AUC	ROC-AUC
Logistic Regression	No resampling	0.290	0.009	0.017	0.001	0.175	0.804
Logistic Regression	Class weighting	0.134	0.833	0.232	0.341	0.162	0.805
Logistic Regression	Random oversampling	0.134	0.822	0.231	0.338	0.162	0.803
Logistic Regression	SMOTE (SMOTENC)	0.139	0.511	0.219	0.201	0.143	0.760
Logistic Regression	Random undersampling	0.132	0.821	0.227	0.344	0.158	0.795
Decision Tree	No resampling	0.371	0.028	0.052	0.003	0.190	0.809
Decision Tree	Class weighting	0.126	0.891	0.221	0.393	0.178	0.797
Decision Tree	Random oversampling	0.129	0.883	0.225	0.381	0.190	0.797
Decision Tree	SMOTE (SMOTENC)	0.131	0.663	0.218	0.281	0.156	0.780
Decision Tree	Random undersampling	0.128	0.843	0.222	0.365	0.152	0.787
Random Forest	No resampling	0.000	0.000	0.000	0.000	0.245	0.830
Random Forest	**Class weighting**	**0.180**	**0.621**	**0.279**	**0.180**	**0.238**	**0.833**
Random Forest	Random oversampling	0.291	0.196	0.233	0.031	0.243	0.835
Random Forest	SMOTE (SMOTENC)	0.170	0.293	0.215	0.091	0.176	0.797
Random Forest	Random undersampling	0.128	0.928	0.224	0.404	0.193	0.807

Without imbalance handling, all three classifiers almost never predicted fraud at the 0.5 threshold. The Random Forest did not flag a single claim as fraudulent in any fold, and Logistic Regression caught less than 1% of fraud cases. This is the expected behaviour of a classifier trained on data where 94% of the examples belong to one class: the estimated probability of fraud rarely goes above 0.5.

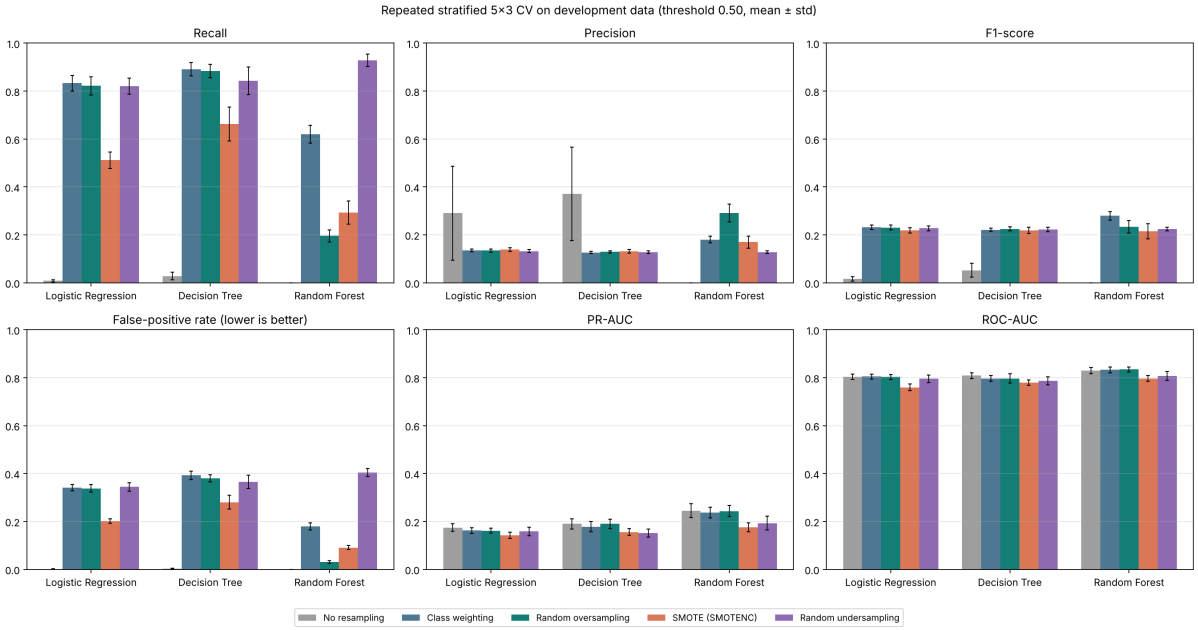


Figure 3: Cross-validation results by model and imbalance strategy (mean ± standard deviation, threshold 0.5).

5.2 Effect of imbalance handling

To check whether the improvements were consistent, we compared each strategy with the same model without imbalance handling, fold by fold. Every one of the 12 model and strategy combinations improved recall and F1-score in all 15 folds. Recall increased by between 0.20 and 0.93 and F1-score by between 0.17 and 0.28.

The cost of this improvement is a higher false-positive rate, which also rose in every fold. Strategies that caught the most fraud also flagged the most legitimate claims. For example, Random Forest with random undersampling had the highest recall (0.928) but also the highest FPR (0.404), while Random Forest with random oversampling had a much lower FPR (0.031) but caught only 19.6% of fraud. Class weighting and random oversampling gave very similar results for Logistic Regression, which makes sense because duplicating minority examples has a similar effect to weighting them.

5.3 SMOTE compared with the baseline

Table 5 compares SMOTE with no resampling.

Table 5: SMOTE (SMOTENC) compared with no resampling (cross-validation mean, threshold 0.5).

Model	Recall	F1-score	FPR	PR-AUC
Logistic Regression	0.009 → 0.511	0.017 → 0.219	0.001 → 0.201	0.175 → 0.143
Decision Tree	0.028 → 0.663	0.052 → 0.218	0.003 → 0.281	0.190 → 0.156
Random Forest	0.000 → 0.293	0.000 → 0.215	0.000 → 0.091	0.245 → 0.176

SMOTE clearly improved fraud detection over the baseline for all three models. However,

it was not the best strategy for any of them. For each model, its F1-score was the lowest of the four imbalance-handling techniques, and it reduced PR-AUC and ROC-AUC in every fold. One likely reason is that most of our features are categorical.

5.4 Selected model

The best strategy for each model, by cross-validated F1-score, was class weighting for Logistic Regression (F1 = 0.232), random oversampling for the Decision Tree (0.225) and class weighting for the Random Forest (0.279). The **Random Forest with class weighting** had the highest F1-score overall, as well as the highest precision among the high-recall candidates, and was selected.

5.5 Results on the test set

Table 6 gives the test results for the selected model, compared with the Random Forest without imbalance handling.

Table 6: Test-set results for the selected model (threshold 0.5), with 95% bootstrap confidence intervals.

Metric	RF + class weighting	95% CI	RF, no resampling	Change (95% CI)
Precision	0.184	0.162 – 0.206	0.000	+0.184 (+0.162 to +0.206)
Recall	0.595	0.519 – 0.665	0.000	+0.595 (+0.519 to +0.665)
F1-score	0.281	0.249 – 0.313	0.000	+0.281 (+0.249 to +0.313)
False-positive rate	0.168	0.155 – 0.180	0.000	+0.168 (+0.155 to +0.180)
PR-AUC	0.193	0.166 – 0.237	0.215	−0.022 (−0.049 to +0.003)
ROC-AUC	0.829	0.803 – 0.851	0.824	+0.006 (−0.004 to +0.015)

The test results agree closely with the cross-validation estimates (F1-score 0.281 against 0.279), which suggests the model was not overfitted to the development data. The selected model detected 110 of the 185 fraudulent claims. The confidence intervals for recall and F1-score are well above zero, so the improvement over the baseline is not due to chance. The other candidates also behaved on the test set as in cross-validation. For example, Logistic Regression with class weighting had a test recall of 0.838 at an FPR of 0.342, and the Decision Tree with random oversampling had a recall of 0.870 at an FPR of 0.359. The test results for all 15 candidates are included in our output files.

Figure 4 shows the confusion matrices. Without imbalance handling, Logistic Regression, the Decision Tree and the Random Forest caught only 2, 9 and 0 of the 185 fraud cases respectively. With imbalance handling, Logistic Regression and the Decision Tree caught more fraud than the Random Forest (155 and 161 cases), but they wrongly flagged around 1,000 legitimate claims each, against 487 for the Random Forest.

Figure 5 shows that the precision-recall and ROC curves with and without imbalance handling are very close to each other for each model. We return to this point in Section 6.

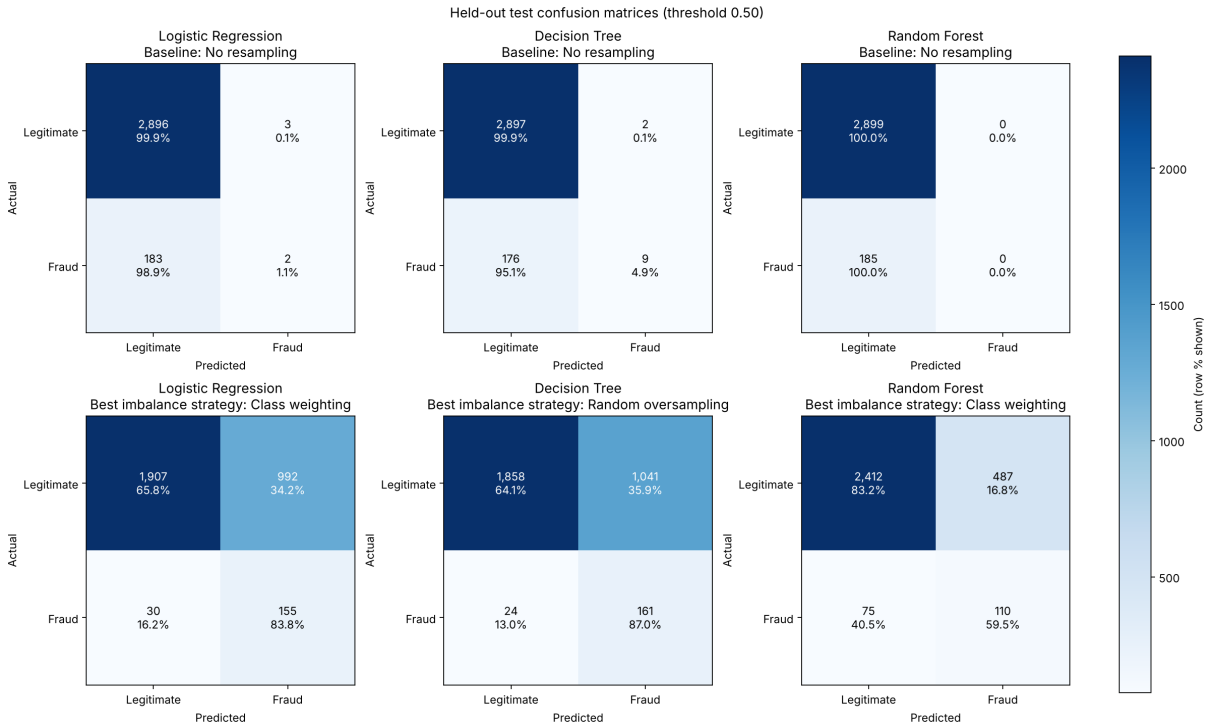


Figure 4: Test-set confusion matrices at threshold 0.5. Top row: no imbalance handling. Bottom row: the best strategy for each model.

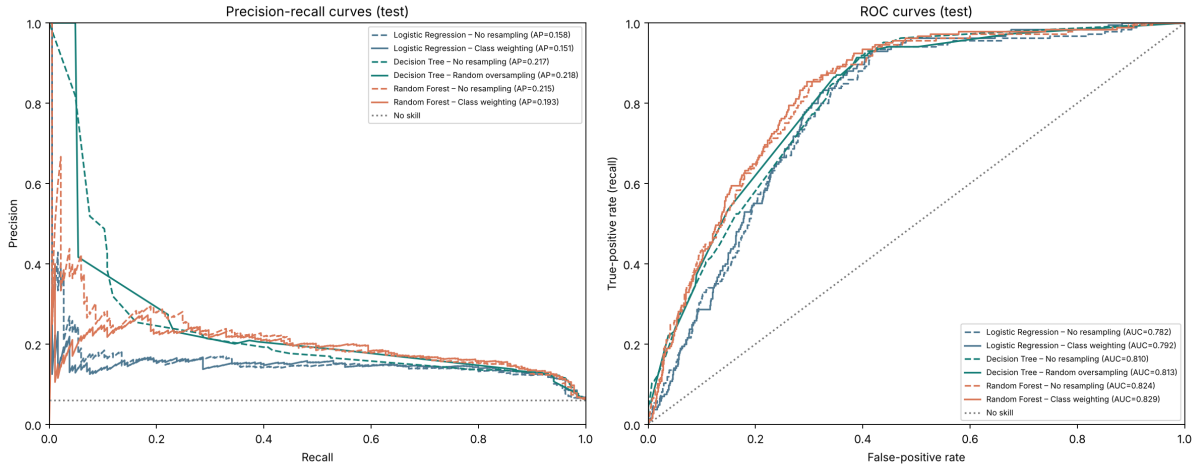


Figure 5: Precision-recall and ROC curves on the test set (dashed: no imbalance handling; solid: best strategy for each model).

6 Discussion

Our results show two different effects of imbalance handling. At the default threshold, the improvement is large and consistent: recall and F1-score increased in every fold, and their confidence intervals on the test set are well above zero. However, the threshold-free metrics did not improve. For the selected model, PR-AUC changed by -0.022 and ROC-AUC by $+0.006$ on the test set, and both confidence intervals include zero. In cross-validation, 11 of the 12 model and strategy combinations had a lower mean PR-AUC than the same model without imbalance handling. The Random Forest without resampling actually had the highest cross-validated PR-AUC of all candidates (0.245), even though it never predicted fraud at the 0.5 threshold.

This means that imbalance handling mainly shifted the predicted probabilities of fraud upwards, so that more claims crossed the 0.5 threshold. It did not make the models better at ranking fraudulent claims above legitimate ones. We therefore cannot claim that imbalance handling improved the models' underlying ability to separate the two classes; what it improved is their classification at the standard threshold. Figure 5 shows the same thing: the curves with and without imbalance handling are almost on top of each other.

7 Conclusion

In this project we compared three classifiers and four imbalance-handling techniques for detecting fraudulent vehicle insurance claims, a problem where only 6% of claims are fraudulent. Without imbalance handling, none of the models was useful at the standard 0.5 threshold, as they almost never predicted fraud. All four techniques (class weighting, random oversampling, SMOTE and random undersampling) improved recall and F1-score consistently, at the cost of more false positives.

The best overall combination was a Random Forest with class weighting. On the held-out test set it detected 59.5% of fraudulent claims with an F1-score of 0.281 and a false-positive rate of 16.8%. SMOTE, implemented with SMOTENC because most features are categorical, improved on the baseline but was the weakest of the four techniques on this dataset.

We also found that imbalance handling improved classification at the default threshold without clearly improving the models' ranking ability (PR-AUC and ROC-AUC). In practice, the choice of model should depend on how many flagged claims an insurer can investigate. Precision is at most about 20% for our models, so any fraud detection system based on these features would be best used to prioritise claims for review rather than to make final decisions.

Appendix: Supporting files and reproducibility

The following supporting material accompanies this report:

- **Analysis script** (`New_Insurance_C.py`): Python code for the full analysis, covering

data cleaning, exploratory analysis, cross-validation, model selection, test evaluation, and all figures and tables.

- **Dataset** (`vehicle_insurance.csv`): a copy of the public Kaggle dataset, placed next to the analysis script.
- **Requirements file** (`requirements.txt`): the Python packages needed to run the script.
- **Output folders** (`eda_outputs/` and `model_outputs/`): all tables (CSV), figures and text summaries produced by the script, including the test results for every candidate model.
- **README file** (`README.txt`): instructions for running the code.

The dataset is publicly available from `https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection` (file `fraud_oracle.csv`), and a copy is included with the supporting material.

To reproduce the results:

1. Install Python 3 and the packages in `requirements.txt`:
   `pip install -r requirements.txt`
2. Make sure the dataset is in the same folder as the script. The included `vehicle_insurance.csv` is already there; a fresh download of `fraud_oracle.csv` from Kaggle also works without renaming.
3. Run `python New_Insurance_C.py`.

All random steps use a fixed seed (2026), so the results are reproducible. The full run took about 12 minutes on a 2-core machine. The results in this report were produced with Python 3.13, pandas 3.0.5, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1, imbalanced-learn 0.14.2 and Matplotlib 3.11.2.